数字逻辑基础与Verilog设计
---组合篇

第4章 组合电路模块

本章内容

- 多路选择器（4.1）
- 译码器、多路分配器（4.2）
- 编码器、代码转换器与算术比较电路（4.3-4.5）
  - 用Verilog设计上述组合电路模块（4.6）
- 算术运算电路（3.4，…）

6. 编码器

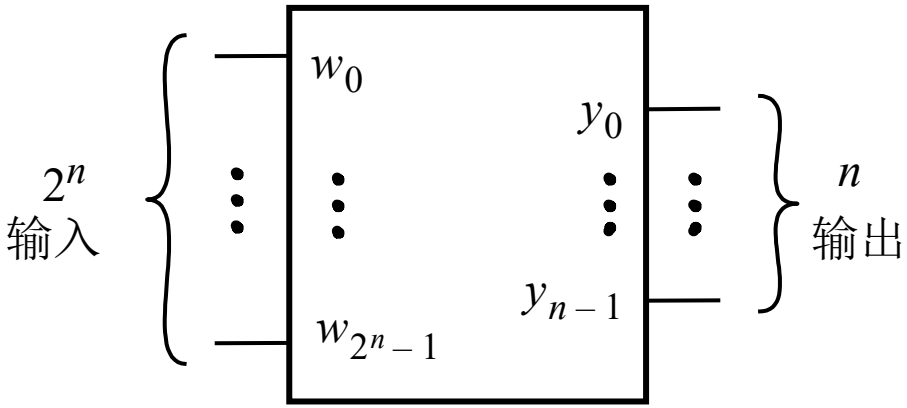
将某一特定的逻辑信号变换为二进制代码；能够实现编码功能的逻辑部件称为编码器，与译码器功能相反。

- 用途：减少表示给定信息的代码长度。
- 例如：病房呼叫器
- 分为：二进制编码器；优先编码器；

6. 编码器

二进制编码器：

- 将2^n个输入信息编码成n位代码输出。在任何时刻只允许一个输入信号（待编码信号）为1，用一个二进制数（编码）作为输出来确定哪一个信号输入，叫做2^n~n编码器
- 特点：输入是独热码，存在无关项。



w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

真值表(4-to-2 二进制编码器)

图 4.18. 2^n-to-n二进制编码器

6. 编码器

二进制编码器的Verilog HDL实现：

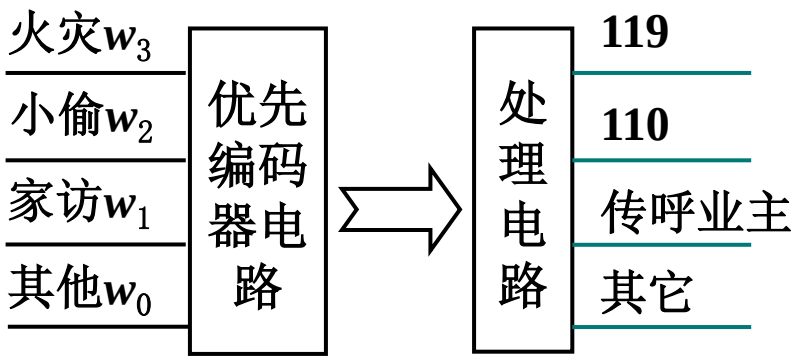
```
module encoder_8_3(
 input [7:0] in,
 output [2:0] out
);
assign out = ({3{in[0]}} & 3‘d0)
                | ({3{in[1]}} & 3’d1)
                | ({3{in[2]}} & 3‘d2)
                | ({3{in[3]}} & 3’d3)
                | ({3{in[4]}} & 3‘d4)
                | ({3{in[5]}} & 3’d5)
                | ({3{in[6]}} & 3‘d6)
                | ({3{in[7]}} & 3’d7);

endmodule
```

6. 编码器

优先编码器：

- 优先编码器可以同时输入几个信号，但在设计时已经将各输入信号的优先顺序排好。当几个信号同时输入时，优先权最高的信号优先编码。
- 例：家用险情检测器



w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

图4.20. 4-to-2 优先编码器

6. 编码器

优先编码器的Verilog HDL实现：

```
module priority_8_3 (
    input  [7:0] in;
    output [2:0] out
);
assign out = in[7] ? 3'd7 :
             in[6] ? 3'd6 :
             in[5] ? 3'd5 :
             in[4] ? 3'd4 :
             in[3] ? 3'd3 :
             in[2] ? 3'd2 :
             in[1] ? 3'd1 :
                     3'd0 ;
endmodule
```

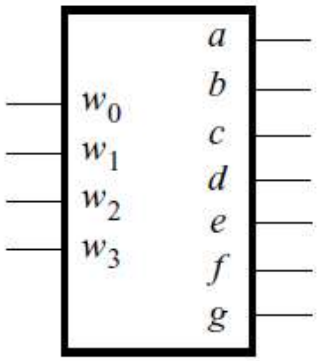
其中，**in[7]**优先级最高，可以改写为**if-else**代码。

教材图 **4.36**和图**4.38**，分别采用**for**语句和**casex**语句实现优先编码器，自行学习。

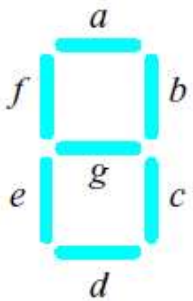
7. 代码转换器

16进制到7段显示代码转换器：

- 数字0到9：BCD-7段译码器.
- 数字10-15：显示A，b，C，d，E，F.



(a) Code converter



(b) 7-segment display

w_3	w_2	w_1	w_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	1	1	0	1	1	1
1	0	1	1	0	0	1	1	1	1	1
1	1	0	0	1	0	0	1	1	1	0
1	1	0	1	0	1	1	1	1	0	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	1	1	1

(c) Truth table

图 4.21. A hex-to-7-segment display code converter.

7. 代码转换器

代码转换器的Verilog实现：

```
module seg7 (
    input [3:0] hex，
    output reg [1:7] leds
);

    always @(hex)
        case (hex)
          0: leds = 7’b1111110;
          1: leds = 7’b0110000;
          2: leds = 7’b1101101;
          3: leds = 7’b1111001;
          4: leds = 7’b0110011;
          5: leds = 7’b1011011;
          6: leds = 7’b1010111;
          7: leds = 7’b1110000;
          8: leds = 7’b1111111;
          9: leds = 7’b1111011;
         10: leds = 7’b1110111;
         11: leds = 7’b0011111;
         12: leds = 7’b1001110;
         13: leds = 7’b0111101;
         14: leds = 7’b1001111;
         15: leds = 7’b1000111;
        endcase

endmodule
```

图 4.34 Code for a hex-to-7-segment decoder

7. 代码转换器

BCD码到格雷码代码转换器：

– 格雷码：
  - 特点：任何相邻的两个代码间仅有一位不同。
  - 是一种错误最小化的编码方式；
  - 是一种循环码：
  - 转换方法：从BCD码最右一位起，依次将每一位与左边一位异或，作为对应格雷码该位的值，最左边一位不变。

$$g_3 = b_3, \quad g_0 = b_0 \oplus b_1,$$

$$g_1 = b_1 \oplus b_2, \quad g_2 = b_2 \oplus b_3$$

b_3	b_2	b_1	b_0	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

(a) Truth table

7. 代码转换器

代码转换器的Verilog实现：

```
module Bcd2Gray1 (b, g);
    input [3:0] b;
    output reg [3:0] g;

    always @(b)
        case (b)
          0: g = 4'b0000;
          1: g = 4'b0001;
          2: g = 4'b0011;
          3: g = 4'b0010;
          4: g = 4'b0110;
          5: g = 4'b0111;
          6: g = 4'b0101;
          7: g = 4'b0100;
          8: g = 4'b1100;
          9: g = 4'b1101;
         10: g = 4'b1111;
         11: g = 4'b1110;
         12: g = 4'b1010;
         13: g = 4'b1011;
         14: g = 4'b1001;
         15: g = 4'b1000;
        endcase
endmodule
```

//行为级描述

```
module Bcd2Gray2 (b, g);
    input [3:0] b;
    output reg [3:0] g;

    always @(b)
        begin
            g[0] =b[0]^b[1];
            g[1] =b[1]^b[2];
            g[2] =b[2]^b[3];
            g[3] =b[3];
        end
endmodule
```

Figure. A BCD-to-GRAY code converter.

7. 代码转换器

代码转换器的Verilog实现：

```
module Bcd2Gray3 (b, g);
    parameter length = 4;
    input [length-1:0] b;
    output reg [length-1:0] g;
    integer k;

    always @(b)
        begin
            for(k=0;k<length-1;k=k+1)
                g[k] =b[k]^b[k+1];
            g[k] =b[k];
        end
endmodule
```

```
// 数据流描述
module Bcd2Gray4 (b, g);
    parameter length = 4;
    input [length-1:0] b;
    output [length-1:0] g;

    assign g=(b>>1)^b;

endmodule
```

Figure. A BCD-to-GRAY code converter.

8. 数值比较器

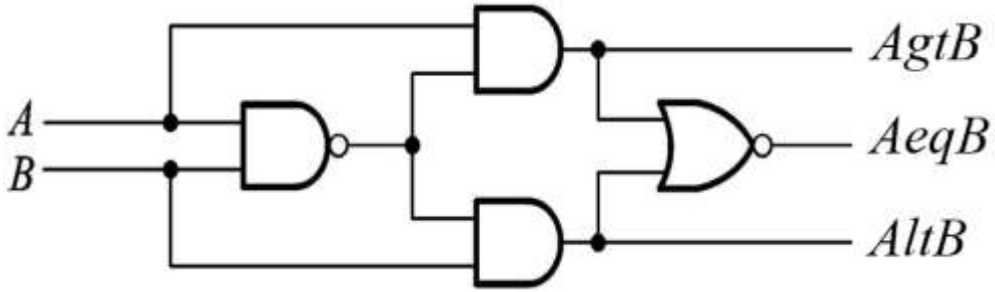
数值比较器：

- 输入为两个n位无符号二进制数A，B；
- 输出为3个信号，属于独热码：
  - 当*A*=*B*时，*AeqB*=1；
  - 当*A*>*B*时，*AgtB* =1；
  - 当*A*<*B*时，*AltB* =1。
- 一位数值比较器：

$*A > B$ 则$AB'=1$，$\therefore AgtB = AB'$

$*A < B$ 则$A'B=1$，$\therefore AltB = A'B$

$*A = B$ $\therefore AeqB = A \odot B$



8. 数值比较器

数值比较器：

- 4位比较器

$A=a_3a_2a_1a_0$, $B=b_3b_2b_1b_0$				输出		
a_3b_3	a_2b_2	a_1b_1	a_0b_0	*AgtB*	*AeqB*	*AltB*
>	×	×	×	1	0	0
=	>	×	×	1	0	0
=	=	>	×	1	0	0
=	=	=	>	1	0	0
=	=	=	=	0	1	0
<	×	×	×	0	0	1
=	<	×	×	0	0	1
=	=	<	×	0	0	1
=	=	=	<	0	0	1

$$AgtB = a_3b_3' + (a_3 \oplus b_3)' a_2 b_2' + (a_3 \oplus b_3)'(a_2 \oplus b_2)' a_1 b_1'$$
$$+ (a_3 \oplus b_3)'(a_2 \oplus b_2)'(a_1 \oplus b_1)' a_0 b_0'$$

$$AeqB = (a_3 \oplus b_3)'(a_2 \oplus b_2)'(a_1 \oplus b_1)'(a_0 \oplus b_0)'$$

$$AltB = (AeqB + AltB)'$$

8. 数值比较器

数值比较器的Verilog实现：

```
module compare (A, B, AeqB, AgtB, AltB);
    input [3:0] A, B;
    output reg AeqB, AgtB, AltB;

    always @(A, B)
    begin
        AeqB = 0;
        AgtB = 0;
        AltB = 0;
        if (A == B)   AeqB = 1;
        else if (A > B)
            AgtB = 1;
        else
            AltB = 1;
    end

endmodule
```

图 4.40. Verilog code for a four-bit comparator.

本章作业：

- **4.1（或4.2），4.3（或4.4），4.5，4.8，4.11；**
- **4.16，4.18，4.19，4.21，4.22，4.25，**
- **4.26，4.27，**
- **4.29，4.30，4.32，4.33**